BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 6

Roll no: 44 PRN No: 12410016 Name of Student: **Sahil Namdeo Tagade**

Subject Name : Deep Learning

Problem Statement

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset (PlantVillage dataset), and evaluate its performance on unseen validation data.

Algorithm

1. Start.

2. Import the required libraries: TensorFlow/Keras (ImageDataGenerator, Sequential, Conv2D, MaxPooling2D, Flatten, Dense, Dropout), Matplotlib, NumPy, and the Kaggle API for dataset access.

3. Configure the Kaggle API credentials (username and key) and search the Kaggle platform for the PlantVillage plant-disease dataset.

4. Download the PlantVillage dataset (emmarex/plantdisease) using the Kaggle API and extract the downloaded ZIP archive into a working 'dataset' directory.

5. Locate the dataset folder and identify the disease classes present; the extracted dataset contains 15 classes covering Pepper, Potato, and Tomato leaf images, both healthy and diseased.

6. Perform image preprocessing using ImageDataGenerator: rescale pixel values (1./255) and reserve 20% of the images for validation using validation_split.

7. Generate the training data batches using flow_from_directory() with target image size 128×128, batch size 32, categorical class mode, and the 'training' subset.

8. Generate the validation data batches using flow_from_directory() with the same parameters and the 'validation' subset.

9. Visualize a batch of sample training images in a 3×3 grid to inspect the dataset.

10. Build the CNN architecture using Sequential(): a first convolution layer (32 filters, 3×3 kernel, ReLU activation, input shape 128×128×3) followed by max pooling (2×2); a second convolution layer (64 filters, 3×3 kernel, ReLU) followed by max pooling (2×2); a flatten layer; a fully connected dense layer (128 neurons, ReLU); a dropout layer (rate 0.5) for regularization; and an output dense layer (15 neurons, softmax) for the 15 disease classes.

11. Display the model architecture and parameter counts using model.summary().

12. Compile the model with the Adam optimizer, categorical_crossentropy as the loss function, and accuracy as the evaluation metric.

13. Train the model using model.fit() on the training data for 5 epochs, performing forward propagation and backpropagation while validating on the validation data after every epoch.

14. Plot the training vs validation accuracy curve against epochs to visualize the learning behaviour.

15. Plot the training vs validation loss curve against epochs to check for overfitting or underfitting.

16. Evaluate the trained model on the validation set using model.evaluate() to obtain the validation loss and validation accuracy.

17. Generate a prediction on a sample validation image using model.predict(), convert the softmax probability output to a class label using np.argmax(), and compare it against the actual class.

18. Save the trained model to disk (PlantVillage_CNN_Model.h5) for future reuse.

19. List and enumerate all 15 disease classes present in the dataset.

20. Visualize a grid of sample images across the classes, and display one representative sample image from each of the 15 classes for a complete overview of the dataset.

21. Stop.

Code

```
# Design and implement a Convolutional Neural Network (CNN) for image classification using
the Tomato or Soybean disease dataset
```

```
#KGAT_363734446cb801ff5c6cef4aba143ed3
```

```
!pip install -q kaggle
```

```
import os

os.environ["KAGGLE_USERNAME"] = "namdeosahil"
os.environ["KAGGLE_KEY"] = "KGAT_363734446cb801ff5c6cef4aba143ed3_HERE"

print("Kaggle credentials configured successfully!")
```

```
!kaggle datasets list -s plantdisease
```

```
!kaggle datasets download -d emmarex/plantdisease
```

```
#importing libraries

import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense, Dropout

print("Libraries Imported Successfully!")
```

```
#extracting the dataset

import zipfile

with zipfile.ZipFile("plantdisease.zip", "r") as zip_ref:
    zip_ref.extractall("dataset")

print("Dataset Extracted Successfully!")
```

```
import os
os.listdir("dataset")
```

```
#dataset path

dataset_path = "dataset/PlantVillage"
classes = os.listdir(dataset_path)

print("Total Classes :", len(classes))
print(classes)
```

```
#image processing

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
```

```
#training data

train_data = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(128,128),
    batch_size=32,
    class_mode="categorical",
    subset="training"
)
```

```
#validation data

validation_data = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(128,128),
    batch_size=32,
    class_mode="categorical",
    subset="validation"
)
```

```
#displaying sample images

images, labels = next(train_data)
plt.figure(figsize=(10,10))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(images[i])
    plt.axis("off")

plt.show()
```

```
#building cnn model

model = Sequential()

# First Convolution Layer
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(pool_size=(2,2)))

# Second Convolution Layer
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten Layer
model.add(Flatten())

# Fully Connected Layer
model.add(Dense(128, activation='relu'))

# Dropout Layer
model.add(Dropout(0.5))

# Output Layer (15 Classes)
model.add(Dense(15, activation='softmax'))

print("CNN Model Created Successfully!")
```

```
#model summary

model.summary()
```

```
#compiling the model

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model Compiled Successfully!")
```

```
#training the model

history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=5
)
```

```
#accuracy graph
```

```
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid(True)
plt.show()
```

```
#loss graph

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid(True)
plt.show()
```

```
#model evaluation

loss, accuracy = model.evaluate(validation_data)

print("Validation Loss :", loss)
print("Validation Accuracy :", round(accuracy*100,2), "%")
```

```
#prediction

images, labels = next(validation_data)
prediction = model.predict(images)

predicted_class = np.argmax(prediction[0])
actual_class = np.argmax(labels[0])

print("Actual Class :", actual_class)
print("Predicted Class :", predicted_class)

plt.imshow(images[0])
plt.axis("off")
plt.show()
```

```
#save model

model.save("PlantVillage_CNN_Model.h5")

print("Model Saved Successfully!")
```

```
import os

dataset_path = "dataset/PlantVillage"

classes = sorted(os.listdir(dataset_path))

print("Total Classes:", len(classes))

for i, cls in enumerate(classes, start=1):
    print(f"{i}. {cls}")
```

```
import os
import matplotlib.pyplot as plt
from PIL import Image

dataset_path = "dataset/PlantVillage"

plt.figure(figsize=(15,15))
count = 1

for cls in sorted(os.listdir(dataset_path)):
    class_path = os.path.join(dataset_path, cls)
    for image in os.listdir(class_path):
        img = Image.open(os.path.join(class_path, image))
        plt.subplot(5,5,count)
        plt.imshow(img)
        plt.title(cls, fontsize=7)
        plt.axis("off")
        count += 1
        if count > 25:
            break
    if count > 25:
        break

plt.tight_layout()
plt.show()
```

```
#sample image from each class

import os
import matplotlib.pyplot as plt
from PIL import Image

dataset_path = "dataset/PlantVillage"
```

```
# Get all class names and sort them
classes = sorted(os.listdir(dataset_path))

# Create figure
plt.figure(figsize=(18, 15))

# Loop through each class
for i, cls in enumerate(classes):
    class_path = os.path.join(dataset_path, cls)

    # Get the first image from the class
    image_name = os.listdir(class_path)[0]
    image_path = os.path.join(class_path, image_name)

    # Open image
    img = Image.open(image_path)

    # Display image
    plt.subplot(4, 4, i + 1)
    plt.imshow(img)
    plt.title(cls.replace("_", " "), fontsize=9)
    plt.axis("off")

plt.suptitle("One Sample Image from Each Plant Disease Class", fontsize=18)
plt.tight_layout()
plt.show()
```

Output

The following screenshots show the executed notebook cells along with their corresponding outputs, plots, and results.

```
# Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset

#KGAT_363734446cb801ff5c6cef4aba143ed3
```

```
!pip install -q kaggle
```

```
import os

os.environ["KAGGLE_USERNAME"] = "namdeosahil"
os.environ["KAGGLE_KEY"] = "KGAT_363734446cb801ff5c6cef4aba143ed3_HERE"

print("Kaggle credentials configured successfully!")
```

```
Kaggle credentials configured successfully!
```

```
!kaggle datasets list -s plantdisease
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest versio
ref                                          title                                          size  lastUpdated                 d
-------------------------------------------  ---------------------------------------------  ----------  --------------------------  -
mummidi31/plantdisease                       Plantdisease                                        12215  2020-12-14 06:53:29.700000
ly9802/plantdisease                          PlantDisease                                    855079835  2021-12-10 05:52:35.963000
emmarex/plantdisease                         PlantVillage Dataset                            689512690  2018-10-30 01:16:23.440000
minhaz027/plantdisease                       Plantdisease                                    153905669  2021-04-25 20:53:24.067000
sayanroy058/plantdisease                     PlantDisease                                    934938040  2024-07-23 18:41:45.293000
ahmedsharaf09/plantdiseasegp                 PlantDisease(GP)                                925469812  2025-02-01 01:39:43.760000
zebraeatcake/plamtdisease                    plantdisease                                    795294136  2025-07-01 09:48:04.110000
codemaivanngu/plantdisease-sg2ada            PlantDisease StyleGAN2-ADA Generated Images  4540020802  2026-01-05 06:32:17.780000
nghido875/plantdisease                       plantdisease                                    665677599  2024-05-23 15:44:05.050000
bhavishyawefarwef/bhavishya-plantdisease     Bhavishya plantdisease                          758878745  2026-01-16 11:12:33.973000
saroz014/plantdisease                        Plant Disease                                   605040014  2018-09-16 07:28:51.120000
varshiniramanji/dataset-123                  Dataset_123                                         16252  2026-08-05 14:53:43.227000
rithwikvattipally/crop-disease               Crop-Disease                                         8145  2026-05-10 08:07:48.627000
rizkyarya/tomatoleafdisease                  tomatoleafdisease                               261698505  2023-05-23 04:57:01.957000
prashant19930517/plantdisease                Plant disease detection                           3850  2023-07-29 17:14:31.003000
haripriyadesai/final-plant-leaves            Final Plant leaves                              901119124  2021-07-06 16:57:51.030000
srinadh1999/plantdisease                     plant-disease                                    25139137  2021-04-18 07:43:36
social888/plantdisease                       Agrivision Dataset                             2109584027  2026-05-08 21:48:51.467000
```

```
!kaggle datasets download -d emmarex/plantdisease
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest versio
Dataset URL: https://www.kaggle.com/datasets/emmarex/plantdisease
License(s): unknown
Downloading plantdisease.zip to /content
100% 658M/658M [00:28<00:00, 23.9MB/s]
```

```
#importing libraries

import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense, Dropout

print("Libraries Imported Successfully!")
```

```
Libraries Imported Successfully!
```

```
#extracting the dataset

import zipfile

with zipfile.ZipFile("plantdisease.zip", "r") as zip_ref:
    zip_ref.extractall("dataset")

print("Dataset Extracted Successfully!")
```

```
Dataset Extracted Successfully!
```

```
import os

os.listdir("dataset")
```

```
['plantvillage', 'PlantVillage']
```

```
#dataset path

dataset_path = "dataset/PlantVillage"

classes = os.listdir(dataset_path)

print("Total Classes :", len(classes))

print(classes)
```

```
Total Classes : 15
['Pepper__bell___Bacterial_spot', 'Tomato_healthy', 'Tomato__Tomato_YellowLeaf__Curl_Virus', 'Tomato_Late_blight', 'Potato___Lat
```

```
#image processing

train_datagen = ImageDataGenerator(

    rescale=1./255,

    validation_split=0.2

)
```

```
#training data

train_data = train_datagen.flow_from_directory(

    dataset_path,

    target_size=(128,128),

    batch_size=32,

    class_mode="categorical",

    subset="training"

)
```

```
Found 16516 images belonging to 15 classes.
```

```
#validation data

validation_data = train_datagen.flow_from_directory(

    dataset_path,

    target_size=(128,128),

    batch_size=32,

    class_mode="categorical",

    subset="validation"

)
```

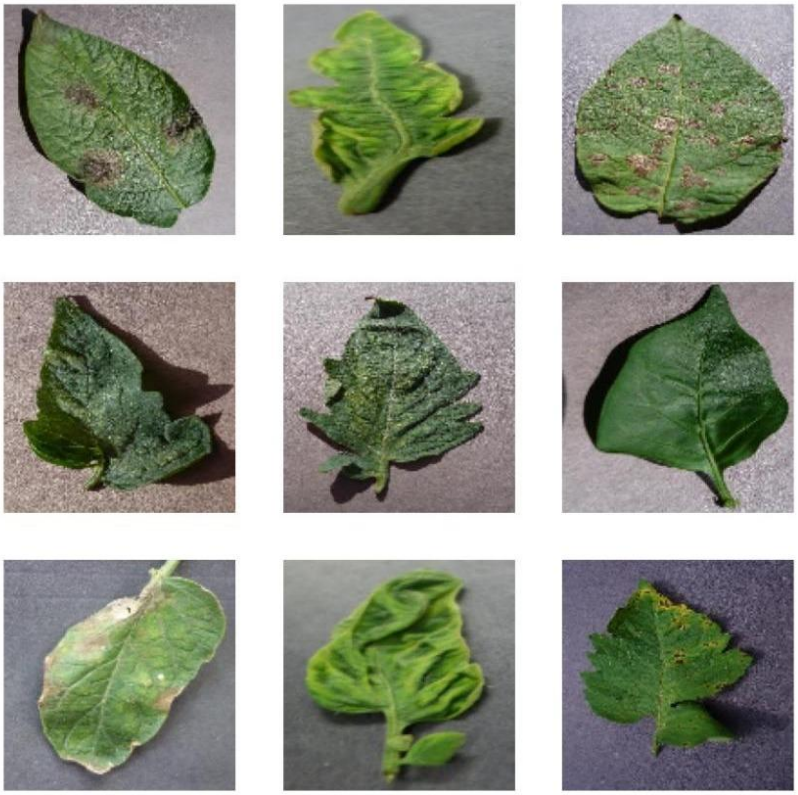
```
Found 4122 images belonging to 15 classes.
```

```
#displaying sample images

images, labels = next(train_data)

plt.figure(figsize=(10,10))
```

```
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(images[i])
    plt.axis("off")
plt.show()
```



```
#building cnn model

model = Sequential()

# First Convolution Layer
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(pool_size=(2,2)))

# Second Convolution Layer
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten Layer
model.add(Flatten())

# Fully Connected Layer
model.add(Dense(128, activation='relu'))

# Dropout Layer
model.add(Dropout(0.5))

# Output Layer (15 Classes)
model.add(Dense(15, activation='softmax'))
```

```
print("CNN Model Created Successfully!")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
CNN Model Created Successfully!
```

```
#model summary

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
flatten (Flatten)	(None, 57600)	0
dense (Dense)	(None, 128)	7,372,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 15)	1,935

Total params: 7,394,255 (28.21 MB)
Trainable params: 7,394,255 (28.21 MB)

```
#compiling the model

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model Compiled Successfully!")
```

```
Model Compiled Successfully!
```

```
#training the model

history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=5
)
```

```
Epoch 1/5
517/517 ━━━━━━━━━━━━━━━━━━━━ 480s 921ms/step - accuracy: 0.5064 - loss: 1.5552 - val_accuracy: 0.7407 - val_loss: 0.7663
Epoch 2/5
517/517 ━━━━━━━━━━━━━━━━━━━━ 475s 918ms/step - accuracy: 0.7028 - loss: 0.8950 - val_accuracy: 0.8054 - val_loss: 0.5973
Epoch 3/5
517/517 ━━━━━━━━━━━━━━━━━━━━ 464s 898ms/step - accuracy: 0.7694 - loss: 0.6776 - val_accuracy: 0.8217 - val_loss: 0.5196
Epoch 4/5
517/517 ━━━━━━━━━━━━━━━━━━━━ 499s 893ms/step - accuracy: 0.8069 - loss: 0.5695 - val_accuracy: 0.8389 - val_loss: 0.4947
Epoch 5/5
517/517 ━━━━━━━━━━━━━━━━━━━━ 464s 897ms/step - accuracy: 0.8317 - loss: 0.4925 - val_accuracy: 0.8617 - val_loss: 0.4189
```

```
#accuracy graph

plt.figure(figsize=(8,5))

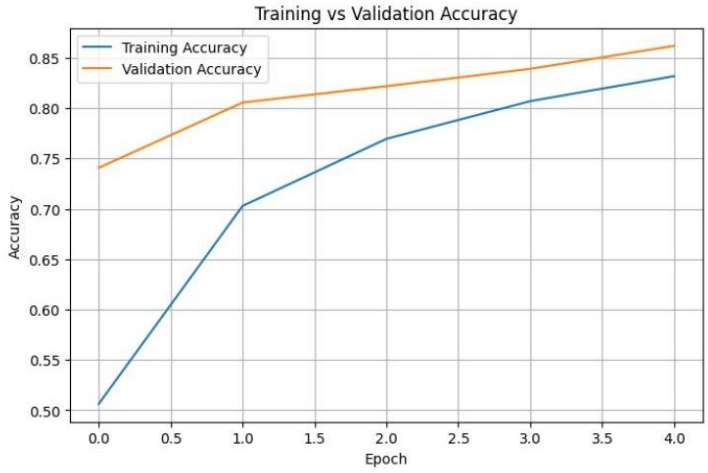
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
```

```
plt.grid(True)

plt.show()
```




```
#loss graph

plt.figure(figsize=(8,5))

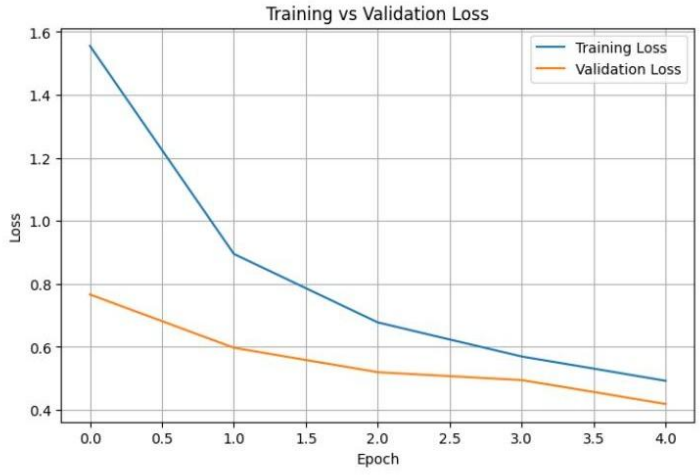
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()
```




```
#model evaluation

loss, accuracy = model.evaluate(validation_data)
```

```
print("Validation Loss :", loss)
print("Validation Accuracy :", round(accuracy*100,2), "%")
```

```
129/129 ━━━━━━━━━━━━━━━━━━━━ 32s 242ms/step - accuracy: 0.8617 - loss: 0.4189
Validation Loss : 0.41887107491493225
Validation Accuracy : 86.17 %
```

```
#prediction

images, labels = next(validation_data)

prediction = model.predict(images)

predicted_class = np.argmax(prediction[0])
actual_class = np.argmax(labels[0])

print("Actual Class :", actual_class)
print("Predicted Class :", predicted_class)

plt.imshow(images[0])
plt.axis("off")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 358ms/step
Actual Class : 1
Predicted Class : 1
```



```
#save model

model.save("PlantVillage_CNN_Model.h5")

print("Model Saved Successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
Model Saved Successfully!
```

```
import os

dataset_path = "dataset/PlantVillage"

classes = sorted(os.listdir(dataset_path))

print("Total Classes:", len(classes))

for i, cls in enumerate(classes, start=1):
    print(f"{i}. {cls}")
```

```
Total Classes: 15
1. Pepper__bell___Bacterial_spot
2. Pepper__bell___healthy
3. Potato___Early_blight
4. Potato___Late_blight
5. Potato___healthy
6. Tomato_Bacterial_spot
7. Tomato_Early_blight
```

```
8. Tomato_Late_blight
9. Tomato_Leaf_Mold
10. Tomato_Septoria_leaf_spot
11. Tomato_Spider_mites_Two_spotted_spider_mite
12. Tomato__Target_Spot
13. Tomato__Tomato_YellowLeaf__Curl_Virus
14. Tomato__Tomato_mosaic_virus
15. Tomato_healthy
```

```
import os
import matplotlib.pyplot as plt
from PIL import Image

dataset_path = "dataset/PlantVillage"

plt.figure(figsize=(15,15))

count = 1

for cls in sorted(os.listdir(dataset_path)):
    class_path = os.path.join(dataset_path, cls)

    for image in os.listdir(class_path):

        img = Image.open(os.path.join(class_path, image))

        plt.subplot(5,5,count)
        plt.imshow(img)
        plt.title(cls, fontsize=7)
        plt.axis("off")

        count += 1

        if count > 25:
            break

    if count > 25:
        break

plt.tight_layout()
plt.show()
```




```
#sample image from each class

import os
```

```
import matplotlib.pyplot as plt
from PIL import Image

dataset_path = "dataset/PlantVillage"

# Get all class names and sort them
classes = sorted(os.listdir(dataset_path))

# Create figure
plt.figure(figsize=(18, 15))

# Loop through each class
for i, cls in enumerate(classes):

    class_path = os.path.join(dataset_path, cls)

    # Get the first image from the class
    image_name = os.listdir(class_path)[0]

    image_path = os.path.join(class_path, image_name)

    # Open image
    img = Image.open(image_path)

    # Display image
    plt.subplot(4, 4, i + 1)
    plt.imshow(img)
    plt.title(cls.replace("_", " "), fontsize=9)
    plt.axis("off")

plt.suptitle("One Sample Image from Each Plant Disease Class", fontsize=18)

plt.tight_layout()

plt.show()
```